Manual sencillo para entender y explicar el código de HeadRace

Guía de estudio para la exposición de Sistemas de Control

Asignatura: Sistemas de Control
Docente: Jorge Ruiz
Integrantes: Rafael Manotas Borja y Julio Sarmiento Peña
Fecha de preparación: lunes 9 de noviembre de 2026
Propósito: comprender el programa y explicarlo con palabras propias el viernes

---

Cómo usar esta guía

No hace falta memorizar cada instrucción del código. La meta es poder explicar el recorrido de la información, qué decisión toma cada parte y qué límites tiene el prototipo. Primero lee el resumen y el diagrama; después estudia el PID, la lógica difusa y el firmware. Al final están el guion corto, las preguntas del profesor y un plan de práctica hasta el viernes.

La idea central es sencilla: **la cámara estima hacia dónde se mueve la cabeza; el ESP32 calcula la posición y la velocidad del auto virtual; el navegador muestra la carrera.** El sensor VL53L0X mide la distancia de la mano para solicitar poderes.

1. El proyecto en un minuto

HeadRace es un juego de carreras controlado con movimientos de la cabeza. La cámara no reconoce órdenes escritas ni necesita marcadores: detecta el rostro y convierte su posición horizontal en un número. Ese número llega al ESP32 por USB serial. El ESP32 ejecuta el PID para calcular la posición lateral del kart y una lógica difusa para decidir la velocidad. El VL53L0X conectado al ESP32 mide la distancia de la mano y permite pedir el escudo o las llamas. La aplicación web recibe esos resultados, mueve el juego en 3D y guarda los datos de las sesiones.

Una explicación breve para el profesor

> *"HeadRace integra visión por computador, control y una interfaz de juego. OpenCV detecta la cara y calcula un desplazamiento horizontal. El backend de Python envía ese dato al ESP32 por USB usando mensajes JSON. En el ESP32 corre el PID que calcula la posición lateral del kart y una lógica difusa que decide la velocidad según el movimiento y la distancia medida. El VL53L0X detecta la mano para solicitar el escudo o las llamas. React y Three.js representan la carrera, los obstáculos y la interfaz; Flask comunica los módulos y SQLite guarda las sesiones."*

Practica esta explicación hasta poder decirla sin leer. Luego puedes profundizar en cada bloque.

2. Qué es físico y qué es virtual

Elemento	Qué hace
Mac y cámara	Capturan imágenes y ejecutan el backend, la interfaz y la escena 3D.
ESP32	Ejecuta el firmware MicroPython con PID, lógica difusa y lectura del sensor.
VL53L0X V2	Mide distancia por tiempo de vuelo de luz; en este proyecto mide la mano para seleccionar poderes.
Auto, pista y obstáculos	Son objetos virtuales dibujados en el navegador con Three.js.

El kart no es un vehículo físico: no hay motores, ruedas motrices ni un controlador de potencia conectados. El ESP32 calcula las decisiones del kart virtual y las comunica al juego. El VL53L0X tampoco reconoce los conos dibujados en pantalla; reconoce objetos reales dentro de su campo de medición, principalmente la mano. Los obstáculos del juego están definidos en el código del frontend.

Esta diferencia es importante para describir el alcance con precisión: se trata de un prototipo de interacción y control de un vehículo simulado, no de un auto real autónomo.

3. Recorrido completo de la información

```
Cámara
  -> OpenCV encuentra el rostro
  -> convierte el centro de la cara en head_x y head_y
  -> Flask envía una línea JSON al ESP32 cada ~50 ms
  -> ESP32 lee el VL53L0X y ejecuta PID + lógica difusa
  -> ESP32 devuelve car_x, car_speed y eventos del sensor
  -> Flask los publica por Socket.IO
  -> React actualiza el kart, el HUD, los obstáculos y los efectos
  -> al terminar o completar un tramo, Flask guarda datos en SQLite
```

Si el ESP32 no está conectado o deja de responder, el backend puede usar el controlador de respaldo en Python. El HUD distingue el control real del modo simulado.

Quién hace cada trabajo

- **OpenCV** encuentra el rostro en la imagen y calcula su centro.
- **Flask** organiza la comunicación entre la cámara, el puerto serial, el juego y la base de datos.
- **ESP32** hace el cálculo de control cuando está conectado.
- **React** arma la página y actualiza la interfaz.
- **React Three Fiber y Three.js** dibujan y animan la escena 3D.
- **Socket.IO** mantiene el canal de mensajes en vivo entre navegador y backend.
- **SQLite** conserva los resultados de las sesiones.

4. Qué significa el mensaje JSON

El backend y el ESP32 intercambian texto, una línea por mensaje. JSON es un formato que representa datos con nombres y valores.

Ejemplo de entrada al ESP32

```
{"head_x":25,"head_y":0,"dt":0.05,"obstacle_dist":2000}
```

Se interpreta así:

- **head_x: 25:** la posición horizontal detectada está hacia la derecha respecto al centro de la imagen.
- **head_y: 0:** la posición vertical está cerca del centro. En el controlador actual, esta coordenada se recibe pero no se usa para mover el kart.
- **dt: 0.05:** han pasado 50 milisegundos desde la muestra anterior.
- **obstacle_dist: 2000:** distancia de respaldo que envía el programa cuando no hay una distancia válida del sensor.

Ejemplo de respuesta

```
{"car_x":8.8,"car_speed":90,"obstacle_dist":2000,"obstacle_alert":false,"sensor_conectado":tru
   e,"sensor_distancia_mm":null,"sensor_mano_detectada":false,"sensor_zona":"reposo","power_ev
   ent":null}
```

- **car_x:** salida del controlador PID, limitada al rango de -100 a 100.
- **car_speed:** decisión de velocidad de la lógica difusa, de 0 a 100. La interfaz la rotula en km/h como valor del juego; no es una medición real de un automóvil.
- **obstacle_alert:** indica si la distancia usada por el control es menor de 400 mm.
- **sensor_conectado:** confirma que el firmware pudo iniciar el VL53L0X.
- **sensor_distancia_mm:** lectura válida en milímetros; null significa que no hubo una lectura dentro del rango aceptado.
- **sensor_zona:** reposo, escudo o llamas, según la distancia.
- **power_event:** se completa una sola vez al reconocer el gesto, con shield o flames.

Una línea JSON por mensaje facilita que MicroPython y Python se entiendan sin depender de un protocolo pesado. El backend abre el puerto serial como único dueño para evitar que dos programas intenten leerlo a la vez.

5. El control PID explicado sin complicaciones

PID significa **Proporcional, Integral y Derivativo**. En HeadRace se usa para que la posición lateral calculada del kart siga el desplazamiento que indica la cabeza sin dar saltos demasiado bruscos.

Piensa que la cabeza propone una posición objetivo y el controlador calcula cómo acercar el kart virtual a esa posición.

- **Error:** diferencia entre la posición objetivo y la posición actual calculada del kart.
- **P, proporcional:** responde al error actual. Si el error crece, la corrección también crece.
- **I, integral:** acumula errores pequeños que persisten. Ayuda a corregir un desajuste sostenido.
- **D, derivativo:** observa cuánto cambia el error entre muestras. Ayuda a moderar cambios repentinos.

La fórmula del programa es:

```
error = objetivo - posicion_actual
integral = integral_anterior + error * dt
derivada = (error - error_anterior) / dt
correccion = Kp * error + Ki * integral + Kd * derivada
```

En el firmware actual:

Parámetro	Valor	Explicación simple
Kp	0.35	Fuerza de respuesta al error presente.
Ki	0.01	Corrección acumulada si queda un error persistente.
Kd	0.02	Reacción al cambio del error.
dt permitido	0.02 a 0.10 s	Evita usar intervalos demasiado cortos o largos.
corrección por muestra	máximo 12 unidades	Limita cuánto cambia el kart en una sola actualización.
posición	-100 a 100	Limita la salida del PID.

El controlador guarda tres datos entre muestras: posición calculada, integral acumulada y error anterior. Esa memoria permite que la parte integral y derivativa tengan sentido.

Qué controla realmente: el PID calcula la posición virtual car_x. No controla un motor físico. Después, el frontend convierte esa salida en un cambio suave entre tres carriles.

La lógica de carriles también importa

El archivo carriles.ts decide cuándo un desplazamiento ya representa una intención de cambiar de carril. En la configuración actual, una señal igual o menor a -14 entra al carril izquierdo; una señal igual o mayor a 10 entra al derecho; dentro de ±8 se considera el centro. Entre esas zonas se conserva el carril anterior para reducir oscilaciones. La función smoothDamp suaviza el movimiento para que el kart no aparezca instantáneamente en otro carril.

6. Lógica difusa explicada con un ejemplo

La lógica difusa sirve cuando las categorías no son simplemente “sí” o “no”. Un movimiento puede ser un poco suave y un poco brusco al mismo tiempo. Una distancia puede quedar entre cerca y media.

El controlador usa dos entradas:

1. **Brusquedad del movimiento de cabeza:** cuánto cambió head_x respecto a la muestra anterior.
2. **Distancia en milímetros:** lectura del sensor o distancia de respaldo del juego.

Cada entrada se clasifica gradualmente. Para el movimiento hay categorías bajo, medio y alto. Para la distancia hay cerca, media y lejos. El controlador combina ambas categorías con nueve reglas:

Movimiento	Obstáculo cerca	Distancia media	Distancia lejos
Bajo	60	90	90
Medio	30	60	90
Alto	30	30	60

Los números son velocidades de referencia para el juego. Si más de una regla se activa parcialmente, el programa calcula un promedio ponderado: las reglas con mayor activación influyen más en el resultado. Por eso el controlador puede dar una velocidad intermedia.

Además, genera una alerta si la distancia usada es menor de **400 mm**. Las funciones de pertenencia y las nueve reglas están en control.py, tanto en el firmware como en la versión Python de respaldo.

Cómo decirlo en la exposición

> *“La lógica difusa permite combinar niveles graduales. Si la cabeza se mueve con mucha brusquedad y la distancia es corta, las reglas reducen la velocidad. Si el movimiento es suave y no hay nada cerca, las reglas permiten una velocidad mayor.”*

La lectura del VL53L0X también se usa como distancia para esta lógica. Como el sensor está orientado a la mano, acercarla puede influir en la decisión de velocidad además de solicitar un poder.

7. ESP32, VL53L0X y los poderes

Los archivos del firmware están en la carpeta firmware/esp32.

Cableado documentado

Módulo VL53L0X	ESP32
SDA	GPIO 21
SCL	GPIO 22
GND	GND
VIN o VCC	Según la tensión admitida por el módulo

La comunicación I2C usa la dirección habitual 0x29. La placa y el módulo deben compartir tierra. El voltaje de alimentación depende del breakout concreto; hay que seguir las marcas y especificaciones de ese módulo.

Qué hace cada archivo del firmware

- **main.py:** programa de arranque, protocolo serial, lectura del sensor, coordinación del PID y la lógica difusa y reconocimiento de gestos.
- **control.py:** implementación MicroPython del PID y la lógica difusa.
- **sensor.py:** crea la conexión I2C, inicializa el VL53L0X, valida la lectura y vuelve a intentar si el sensor falla.
- **vl53l0x_nb.py:** driver de bajo nivel que lee y escribe los registros del sensor por I2C. Deriva de un driver publicado bajo licencia MIT; no es la lógica de HeadRace.
- **backup-before-integration/main.py:** copia del firmware anterior que estaba en la placa antes de la integración.

El firmware anuncia que está listo con un mensaje JSON. Luego espera las muestras del backend, lee el sensor y devuelve una respuesta. El backend espera ese anuncio antes de comenzar a enviar datos.

Distancias y acciones

Distancia de la mano	Acción solicitada
151 a 350 mm, mantenida al menos 300 ms	Solicita escudo.
150 mm o menos	Solicita llamas durante 3 segundos.
450 mm o más, o mano retirada fuera del rango	Rearma el sensor para aceptar otro gesto.

El juego solo acepta el escudo después de **tres obstáculos esquivados en el mismo carril**, y el escudo dura **10 segundos**. Si un obstáculo golpea el kart con el escudo activo, el obstáculo se destruye con un efecto visual y sonido. Si el escudo está activo, el juego bloquea las llamas.

Detalle importante sobre las llamas: en la versión actual, llamas es un efecto visual que aparece detrás del kart durante 3 segundos. No aumenta la velocidad del kart. Si el profesor pregunta por un turbo que acelere, la respuesta honesta es que el código actual representa las llamas, pero todavía no implementa un multiplicador de velocidad.

8. El juego: carriles, obstáculos, choques y escudo

La lógica del juego vive principalmente en carrera.ts y carriles.ts.

- La pista tiene tres carriles y una lista base de obstáculos de tipo cono, caja y barrera.
- El mapa base contiene posiciones definidas en el código. Aunque carriles.ts tiene una función auxiliar para elegir una posición segura aleatoria, la carrera actual utiliza MAPA_BASE; esa función auxiliar no crea los obstáculos de la partida activa.
- El avance de la pista depende de car_speed y del tiempo entre cuadros. Por eso el fondo y los obstáculos parecen acercarse al jugador.
- El kart virtual cambia de carril usando la posición procesada de la cabeza y el suavizado de carriles.ts.
- Si el kart coincide con un obstáculo y no tiene escudo, aumenta el contador de choques. Un choque reinicia la carga de esquivas.
- Para cargar el escudo, el juego registra que el obstáculo estaba en el carril del jugador antes de pasar y que se superó sin choque. Se cuentan esquivas del mismo carril; esquivar cosas de otros carriles no suma.
- Al llegar a tres esquivas, se habilita el escudo. El poder dura 10 segundos.
- Si se superan 10 choques, la carrera termina.
- Los obstáculos destruidos se vuelven a mostrar después de un intervalo para que sigan existiendo durante la carrera.

Los obstáculos se dibujan en Obstaculos.tsx, pero su ubicación y estado se calculan en carrera.ts. Es útil distinguir **la regla** de **la representación**: un módulo decide que hubo choque; otro dibuja el golpe o la explosión.

9. La interfaz y la escena 3D

La aplicación web activa está desarrollada en React y TypeScript. Three.js dibuja la escena; React Three Fiber permite escribir esa escena como componentes React.

Entrada de la aplicación

- **frontend/src/main.tsx** crea la aplicación React y monta el componente principal.
- **frontend/src/App.tsx** crea el Canvas 3D, coloca la cámara y muestra Mundo junto al HUD.
- **frontend/src/ui/Hud.tsx** muestra velocidad, fase del día, cámara, estado del ESP32 y sensor, botones, choques, escudo y resultados.
- **frontend/src/ui/hud.css** da estilo a los controles y paneles.
- **frontend/src/game/carrera.ts** mantiene el estado de la partida, avance, colisiones, esquivas, escudo, llamas, tiempos y comunicación Socket.IO.
- **frontend/src/game/carriles.ts** contiene los tres carriles, posiciones, umbrales de cambio y suavizado.
- **frontend/src/game/audio.ts** carga los audios locales del motor, escudo y explosión. El navegador requiere una acción del usuario para permitir audio; por eso se habilita al iniciar la carrera.

Componentes de la pista

Archivo	Responsabilidad
Mundo.tsx	Une los componentes, configura iluminación, entorno, cámara, ciclo de día y noche y postprocesado.
Cielo.tsx	Dibuja el degradado del cielo y actualiza la dirección del sol según la hora.
Nubes.tsx	Genera una textura procedural de nube y cambia su color según día, tarde o noche.
Pista.tsx	Dibuja el asfalto, líneas, barreras y postes de luz.
Costa.tsx	Dibuja agua, arena, vegetación, palmeras, montañas y faro.
Kart.tsx	Construye y anima el auto, ruedas, luces, llamas y escudo.
Obstaculos.tsx	Modela conos, cajas y barreras y anima sus efectos de destrucción.

Mundo.tsx usa useFrame, un ciclo que se ejecuta en cada cuadro de animación. Ese ciclo avanza la carrera, actualiza la hora ambiental, cambia luces y niebla, mueve la cámara y modifica los efectos. Los componentes usan Mesh y materiales para representar geometría y superficies. El HDRI local secluded_beach_1k.hdr aporta reflejos; el postprocesado incluye SSAO, profundidad de campo, Bloom, ajuste de contraste, aberración cromática y tone mapping.

10. Backend, cámara y almacenamiento

Detección del rostro

detector.py abre la cámara del Mac con OpenCV. Usa el modelo YuNet incluido en la carpeta para detectar rostros. La imagen se voltea horizontalmente para que el movimiento se sienta natural. El centro del rectángulo del rostro se convierte en dos números entre -100 y 100.

Si se deja de ver la cara durante medio segundo, el detector devuelve la condición sin rostro y el control puede volver a una posición neutra. La imagen pequeña con el rectángulo se envía a la interfaz para mostrar el estado de la cámara.

Servidor y comunicación

app.py inicia Flask y Socket.IO. Mantiene tres tareas de fondo:

1. **Captura:** obtiene imágenes, detecta la cara y actualiza head_x y head_y.
2. **Control de respaldo:** ejecuta el controlador Python solo cuando el ESP32 no está tomando el control.
3. **Enlace con el ESP32:** abre el puerto serial, envía JSON y recibe posición, velocidad, lecturas del sensor y gestos.

Un candado protege los datos que comparten las tareas. Cuando llegan datos válidos del ESP32, el backend marca el modo de control como esp32 y no recalcula el movimiento con el controlador Python. Si la placa deja de responder, pasa al modo simulado.

El backend también ofrece rutas HTTP para consultar sesiones y enviar los resultados al almacenamiento. Socket.IO se usa para estado en vivo; HTTP se usa para cargar o guardar historial.

Base de datos

sesiones.py crea y consulta la base SQLite ubicada en data/headrace.sqlite. Guarda identificadores, fecha, duración, vueltas, choques, eventos de obstáculos y velocidad promedio. Valida los datos antes de insertarlos y usa evento_id para no guardar dos veces el mismo evento.

Los datos permanecen en el Mac. La placa no almacena el historial.

11. Mapa de archivos ejecutables

Ruta	Qué debe recordar el expositor
main.py	Arranca el backend de Python.
start-headrace.sh	Inicia backend y frontend de desarrollo; busca un puerto libre del backend entre 5000 y 5009.
headrace/parametros.py	Valores y límites compartidos del controlador de respaldo y servidor.
headrace/captura/detector.py	Cámara, detección facial y conversión del rostro a offsets.
headrace/control/pid.py	PID Python de respaldo.
headrace/control/difusa.py	Reglas difusas Python de respaldo.
headrace/control/paso.py	Combina offsets, PID y lógica difusa en un paso de control.
headrace/backend/app.py	Servidor HTTP, Socket.IO, captura, serial y tareas concurrentes.
headrace/backend/sesiones.py	Validación y consultas de SQLite.
firmware/esp32/main.py	Bucle que corre en el ESP32 y comunica los módulos.
firmware/esp32/control.py	PID y lógica difusa ejecutados en la placa.
firmware/esp32/sensor.py	Lectura y recuperación del sensor ToF.
frontend/src/game/carrera.ts	Estado y reglas de la partida.
frontend/src/game/carriles.ts	Geometría lógica de carriles y transición suave.
frontend/src/game/audio.ts	Motor y efectos de sonido.
frontend/src/scene	Componentes 3D del escenario.
frontend/src/ui	HUD y estilos.
frontend/public/assets/secluded_beach_1k.hdr	Entorno HDR para reflejos de materiales.
frontend/public/audio	Archivos de sonido cargados por la interfaz.

Archivos que pueden confundir

- **headrace/web/juego/** contiene una versión anterior del juego hecha en JavaScript sin React. El arranque actual de la interfaz parte de frontend/index.html y frontend/src/main.tsx. Vite compila frontend a headrace/web/dist para que Flask sirva el resultado. No se deben confundir los archivos antiguos con el frontend activo.
- **frontend/src/game/probar-carriles.ts** es un módulo de comprobación manual de reglas de carriles; no es el punto de entrada de la página.
- **specs/**, README.md, HeadRace_contexto_desarrollo.md e instruccions.md son documentación de contexto y especificación. No se ejecutan durante la partida.
- **node_modules/** y .venv/** son dependencias instaladas, no código propio del juego.

12. Cómo iniciar el proyecto para una demostración

En el directorio raíz del proyecto:

```
./start-headrace.sh
```

El script inicia Flask y Vite. Abre la dirección del juego que imprime la terminal, normalmente http://127.0.0.1:5173/. También imprime el puerto de Flask. El script busca un puerto libre para Flask porque algunos servicios de macOS pueden ocupar el 5000.

Para que el control real funcione, el ESP32 debe tener MicroPython y los cuatro archivos de firmware en su sistema de archivos. Cierra Thonny o cualquier monitor serial antes de iniciar HeadRace; la aplicación debe ser la única que abra el puerto USB. Si no hay placa disponible, el modo de respaldo de Python mantiene parte del juego funcionando.

Las dependencias principales están declaradas en requirements.txt para Python y frontend/package.json para JavaScript. El modelo YuNet de detección facial y los recursos de agua, ambiente, audio y HDRI ya están dentro del proyecto.

13. Guion de demostración

1. Iniciar HeadRace y mostrar el HUD. Señalar si dice “PID + lógica difusa · ESP32” o “Control simulado · ESP32 desconectado”.
2. Iniciar la carrera mirando al frente para fijar el centro.
3. Mover la cabeza a ambos lados y explicar que la cámara genera el dato horizontal, el controlador lo procesa y el kart cambia de carril suavemente.
4. Esquivar tres obstáculos del mismo carril y señalar cómo se carga el escudo. Un choque reinicia esa cuenta.
5. Con el escudo cargado, mantener la mano entre 15 y 35 cm aproximadamente durante 300 ms para solicitarlo. El escudo dura 10 segundos.
6. Con el escudo apagado, acercar la mano a 15 cm o menos para activar las llamas durante 3 segundos. Retirar la mano para rearmar el sensor antes de repetir.
7. Terminar la sesión y mostrar que el resumen e historial quedan en la base de datos.

No describas las llamas como aceleración real: la versión actual las representa visualmente. Si el escudo está activo, el juego bloquea las llamas por diseño.

14. Preguntas que podría hacer el profesor

¿Qué hace el PID aquí?

Calcula la posición lateral del kart virtual a partir del desplazamiento horizontal de la cabeza. Usa error actual, error acumulado y cambio del error para generar una corrección limitada.

¿Por qué usar lógica difusa además del PID?

Resuelven decisiones distintas. El PID sigue la posición lateral. La lógica difusa elige la velocidad a partir de categorías graduales de movimiento y distancia.

¿Dónde se ejecuta el PID real?

En firmware/esp32/control.py, que corre en el ESP32 con MicroPython. headrace/control/pid.py contiene el respaldo del mismo concepto para cuando la placa no responde.

¿La cámara se conecta al ESP32?

No. La cámara se conecta al Mac. OpenCV calcula head_x y head_y y el backend manda esos números al ESP32 por serial.

¿Cómo se conecta el ESP32 con el sensor?

Por I2C, usando SDA y SCL. El firmware usa el driver VL53L0X y expresa las distancias en milímetros.

¿El sensor detecta los obstáculos de la pista?

No detecta los objetos virtuales dibujados en el navegador. Mide objetos reales frente al sensor; en este prototipo esa lectura se usa para reconocer la mano y como entrada de distancia al control difuso.

¿El auto se mueve físicamente?

No. El kart y sus ruedas son una simulación 3D. El ESP32 toma decisiones para ese kart virtual; no hay motores conectados.

¿Qué ocurre si el ESP32 se desconecta?

El backend detecta que no recibe respuestas y puede usar el PID y la lógica difusa de Python. El HUD muestra que el control está simulado.

¿Cómo se comunican los programas?

Con mensajes JSON de una línea por USB serial entre Python y MicroPython; Socket.IO transporta el estado en vivo entre el servidor y el navegador.

¿Qué hace el sensor para activar llamas o escudo?

A 150 mm o menos solicita llamas. Entre 151 y 350 mm, mantenidos 300 ms, solicita el escudo. El juego aplica sus propias condiciones: carga de tres esquivas para el escudo y bloqueo de llamas mientras el escudo esté activo.

¿Dónde se guardan los resultados?

En SQLite, en data/headrace.sqlite, a través de funciones del backend.

¿El valor km/h es una medida real?

No. Es una escala de velocidad del juego producida por la lógica difusa; no se calibra con la velocidad de un vehículo real.

15. Plan corto de estudio hasta el viernes

Día	Qué estudiar	Práctica recomendada
Lunes 9	Diagrama general y propósito de cada módulo.	Explicar el recorrido de datos en 60 segundos.
Martes 10	PID y variables head_x, car_x y dt.	Explicar P, I y D con el ejemplo de seguir una posición.
Miércoles 11	Lógica difusa, nueve reglas, sensor y umbrales.	Practicar las preguntas sobre proximidad y poderes.
Jueves 12	React, Three.js, backend y almacenamiento.	Hacer una demostración completa y responder las preguntas sin leer.
Viernes 13	Repaso ligero.	Presentar el proyecto y reconocer con claridad sus límites.

Divide el tiempo con tu compañero: uno explica la cámara, el backend y el flujo de datos; el otro explica el sensor, el ESP32, el PID y la lógica difusa. Ambos deberían poder describir el sistema completo en pocas frases, porque el profesor puede preguntarle a cualquiera sobre cualquier módulo.

16. Glosario rápido

- **Offset:** distancia numérica respecto al centro de la imagen o de la posición neutral.
- **Setpoint u objetivo:** valor que el controlador intenta alcanzar.
- **Error:** diferencia entre el objetivo y la salida actual.
- **Muestra:** una lectura de entrada y el cálculo que se produce en un instante.
- **dt:** tiempo transcurrido desde la muestra anterior.

- **I2C:** bus de dos señales, SDA y SCL, que permite al ESP32 leer el sensor.
- **USB serial:** canal por el que el Mac y el ESP32 intercambian texto.
- **Socket.IO:** canal de eventos en tiempo real entre backend y navegador.
- **Fallback o respaldo:** controlador alternativo que funciona en Python si el ESP32 no responde.
- **MicroPython:** implementación de Python que puede ejecutarse en microcontroladores compatibles.
- **Three.js:** biblioteca JavaScript para dibujar gráficos 3D en el navegador.
- **React Three Fiber:** integración que permite describir una escena Three.js con componentes React.
- **Vueltas del registro:** el código actual actualiza el contador y guarda un tramo cada 10 segundos; el nombre “vuelta” en la interfaz no significa que se mida una vuelta real de pista delimitada por una línea de meta.

Resumen para recordar

```
La cámara mide el movimiento de la cabeza.
El backend convierte y transporta los datos.
El ESP32 calcula la posición con PID y la velocidad con lógica difusa.
El VL53L0X mide la mano y solicita poderes.
React y Three.js dibujan la carrera.
SQLite conserva los resultados.
```

Si pueden explicar estas seis líneas con sus propias palabras y reconocer qué partes son virtuales, podrán describir el proyecto sin fingir que conocen de memoria cada función.